Name: Akshit Kumar

SAP ID: 590020087

Lab Experiment - 9

Experiment: Classes and Object

Question: Create a class of student (Name, SAP ID, Marks [Physics, Chemistry, Biology]). Create three objects by taking inputs from the user and display details of all students.

Code:

In [3]:
```
class Student:
    # Representing a Student with Name, SAP ID and marks in Physics, Chemistry and Bi
    def __init__(self, Name, SAPID, Marks):
        self.Name = Name
        self.SAPID = SAPID
        self.Marks = Marks

    def displayDetails (self):
        # Displaying the details of the student
        print("-" * 25)
        print(f"Name :{self.Name}")
        print(f"SAP ID :{self.SAPID}")
        print("Marks :")
        for subject, mark in self.Marks.items():
            print(f" {subject}: {mark}")
        total = sum(self.Marks.values())
        totalMarks = len(self.Marks.values())
        print(f"Total Marks: {totalMarks}")
        print("-" * 25)

def studentInput():
    # Prompting the user to enter details for a single student.
    Name = input("Enter student Name: ")
    SAPID = input("Enter student SAP ID: ")
    print("Enter marks for (Name)")
    Physics = float(input("Physics: "))
    Chemistry = float(input("Chemistry: "))
    Biology = float(input("Biology: "))
    Marks = {"Physics": Physics, "Chemistry": Chemistry, "Biology": Biology}
    return Student(Name, SAPID, Marks)

studentList = []

for i in range(3):
    print(f"\nEntering details for Student {i + 1}: ")
    student = studentInput()
    studentList.append(student)

print("\n----- Displaying Details of All Students -----")
for student in studentList:
    student.displayDetails()
```

```
Entering details for Student 1:
Enter marks for Akshit Kumar

------ Displaying Details of All Students ------
-------------------------
Name :Akshit Kumar
SAP ID :590020087
Marks :
 Physics: 82.0
 Chemistry: 75.0
 Biology: 95.0
Total Marks: 253.0
-------------------------
```

Question: Add constructor in the above class to initialize student details of n students and implement following methods:

- Display() student details
- Find Marks_percentage() of each student
- Display result() [Note: if marks in each subject >40% than Pass else Fail]

Write a Function to find average of the class

In [4]:
```
class Student:
    # Constructor to initialize student details
    def __init__(self, name, sap_id, marks):
        self.name = name
        self.sap_id = sap_id
        self.marks = marks  # List: [Physics, Chemistry, Biology]

    # Method to find marks percentage (assuming each subject is out of 100)
    def Marks_percentage(self):
        return sum(self.marks) / 3

    # Method to display result (Pass if all subjects > 40)
    def result(self):
        if all(m > 40 for m in self.marks):
            return "Pass"
        else:
            return "Fail"

    # Method to display student details
    def Display(self):
        print("\nName: {self.name}")
        print("SAP ID: {self.sap_id}")
        print("Marks (P, C, B): {self.marks}")
        print("Percentage: {self.Marks_percentage():.2f}%")
        print("Result: {self.result()}")
        print("-" * 20)

# Function to find the average of the entire class
def class_average(student_list):
    if not student_list:
        return 0
    total_percentage = sum(s.Marks_percentage() for s in student_list)
    return total_percentage / len(student_list)

# Main execution logic
def main():
    n = int(input("Enter the number of students: "))
    students = []

    for i in range(n):
        print("\nEnter details for Student {i+1}:")
        name = input("Student Name: ")
        sap_id = input("Enter SAP ID: ")
        phy = float(input("Enter Physics Marks: "))
        chem = float(input("Enter Chemistry Marks: "))
        bio = float(input("Enter Biology Marks: "))
        students.append(Student(name, sap_id, [phy, chem, bio]))

    print("\n\n---- Displaying All Student Details ----")
    for s in students:
        s.Display()

    avg = class_average(students)
    print("\nOverall Class Average Percentage: {avg:.2f}%")

if __name__ == "__main__":
    main()
```

```
Enter details for Student 1:

---- Displaying All Student Details ----
Name: Akshit Kumar
SAP ID: 590020087
Marks (P, C, B): [82.0, 75.0, 95.0]
Percentage: 84.00%
Result: Pass
--------------------
Overall Class Average Percentage: 84.00%
```

Question: Create a program to implement different types of inheritances.

Code:

In [5]:
```
# Single Inheritance: One child inherits from one parent
class Parent:
    def feature1(self):
        print("Feature 1 from Parent")

class Child(Parent):
    def feature2(self):
        print("Feature 2 from Child")

# Multiple Inheritance: One child inherits from multiple parents
class Father:
    def father_skill(self):
        print("Father's Skill")

class Mother:
    def mother_skill(self):
        print("Mother's Skill")

class MultiChild(Father, Mother):
    def child_skill(self):
        print("Child's unique skill")

# Multilevel Inheritance: A derived class inherits from another derived class
class Grandparent:
    def old_story(self):
        print("Grandparent's story")

class Intermediate(Grandparent):
    def work(self):
        print("Intermediate's work")

class ModernChild(Intermediate):
    def play(self):
        print("Modern Child playing")

# Hierarchical Inheritance: Multiple children inherit from one parent
class Base:
    def info(self):
        print("Base class info")

class SiblingA(Base):
    def task_a(self):
        print("Sibling A task")

class SiblingB(Base):
    def task_b(self):
        print("Sibling B task")

# --- Execution ---
print("\n--- Single Inheritance ---")
s = Child()
s.feature1()
s.feature2()

print("\n--- Multiple Inheritance ---")
m = MultiChild()
m.father_skill()
m.mother_skill()

print("\n--- Multilevel Inheritance ---")
ml = ModernChild()
ml.old_story()
ml.work()
ml.play()

print("\n--- Hierarchical Inheritance ---")
a, b = SiblingA(), SiblingB()
a.info()
b.info()
```

```
--- Single Inheritance ---
Feature 1 from Parent
Feature 2 from Child

--- Multiple Inheritance ---
Father's Skill
Mother's Skill

--- Multilevel Inheritance ---
Grandparent's story
Intermediate's work
Modern Child playing

--- Hierarchical Inheritance ---
Base class info
Base class info
```

Question: Create a class to implement method Overriding.

Code:

In [6]:
```
class Bank:
    # Parent method
    def calculate_interest(self):
        return 0  # Default interest rate

class SBI(Bank):
    # Overriding the parent method
    def calculate_interest(self):
        return 5.5  # SBI specific rate

class HDFC(Bank):
    # Overriding the parent method
    def calculate_interest(self):
        return 6.2  # HDFC specific rate

# Creating objects of child classes
sbi_branch = SBI()
hdfc_branch = HDFC()

# Each object calls its own version of the method
print(f"SBI Interest Rate: {sbi_branch.calculate_interest()}%")
print(f"HDFC Interest Rate: {hdfc_branch.calculate_interest()}%")
```

```
SBI Interest Rate: 5.5%
HDFC Interest Rate: 6.2%
```

Question: Create a class for operator overloading which adds two point objects where point has x and y values.

Code:

In [10]:
```
class Point:
    def __init__(self, x=0, y=0):
        self.x = x
        self.y = y

    # Overloading the + operator
    def __add__(self, other):
        # Creating a new Point object with the sum of coordinates
        new_x = self.x + other.x
        new_y = self.y + other.y
        return Point(new_x, new_y)

    # Method to represent the object as a string
    def __str__(self):
        return f"Point(x={self.x}, y={self.y})"

p1 = Point(10, 20)
p2 = Point(12, 15)

# This calls the __add__ method internally
p3 = p1 + p2

print("P1: {p1}")
print("P2: {p2}")
print("P3 (P1 + P2): {p3}")
```

```
P1: Point(x=10, y=20)
P2: Point(x=12, y=15)
P3 (P1 + P2): Point(x=22, y=35)
```